

1. Introduction:

A machine learning pipeline built to predict freight load rates (posted_rate) for the Freight Rate Prediction assignment. This project trains regression models on 48,000 historical loads (data/train-test.csv, covering January 1 to October 31, 2025) and uses the trained model to:

- score 12,000 held-out loads in validation.csv.
- generate a fixed-scenario December 2025 rate forecast in which every input variable is held constant except the date, isolating the model's purely date-driven behaviour.

2. Key observations from EDA :

- Two columns, **weight** and **market_index**, contained missing values. Missing-value imputation was therefore required during preprocessing.
- The dataset contained **no duplicate records**, so no duplicate removal was required.
- All **load_id** values were unique. Since **load_id** was an identifier rather than a predictive variable, it was excluded from modeling.
- The dataset contained **64 unique pickup locations** and **64 unique delivery locations**. These categorical variables therefore required appropriate encoding before modeling.
- The dataset contained **3 equipment categories**, which required categorical encoding for use in the machine learning models.
- The **date variable ranged from January 1 to October 31, 2025**. Temporal features were therefore extracted from the date variable to capture potential time-related patterns.
- Negative and zero weight values were identified during EDA and flagged as data-quality issues. They were not removed in the final pipeline.
- The geographic coordinates (pickup_lat, pickup_lon, delivery_lat, and delivery_lon) were within valid geographic ranges. Therefore, no treatment was required for invalid geographic values.
- The **distance** variable was right-skewed, with a maximum value of **3,439**. The extreme values appeared potentially legitimate, so they were retained rather than automatically removed.
- The **weight** variable contained extremely large positive values. These extreme positive values required further investigation based on the dataset dictionary and domain context before deciding whether any additional treatment was necessary.
- The **market_index** variable contained no IQR-based outliers. Therefore, no outlier treatment was required for this variable.
- The **quote_signal** variable contained **1,956 observations identified as IQR outliers**, with values outside the bounds of **1.395 and 2.718**. Further investigation showed that these observations were distributed across both tails of the variable and had a systematic

relationship with `posted_rate`. They were therefore considered meaningful observations rather than erroneous extreme values.

- The analysis also revealed a **nonlinear, approximately inverted-U relationship between `quote_signal` and the target variable**. Consequently, the identified `quote_signal` outliers were retained without removal or winsorization.
- Outlier treatment was therefore **variable-specific and domain-based**, rather than relying on automatic removal of all observations identified by the IQR method.
- `posted_rate` was identified as the target variable, making the problem a **supervised regression task**.
- A strong positive linear relationship was observed between `distance` and `posted_rate`, with a correlation coefficient of approximately $r = 0.91$.
- The dataset exhibited **temporal structure**, indicating that date/time-based features and a time-aware validation strategy were appropriate considerations for the modeling process.

3. Data Ingestion:

Dataset overview logging at ingestion time: shape, dtypes, per-column missing counts, duplicate-row count, and descriptive statistics are printed for the raw dataset.

4. Train / Validation / Test Split:

The split is chronological, not random. This is appropriate because `posted_rate` is a time-varying quantity (the December forecasting task specifically requires extrapolating into a future period), so evaluating on the most recent data best simulates how the model will actually be used — predicting rates it has not seen chronologically.

The exact procedure is: sort the full dataset by date ascending, then apply two sequential 80/20 cuts — first separating the most recent 20% as the test set, then splitting the remaining 80% again 80/20 into train and validation. Preprocessing and feature engineering are fitted only after this split is made.

Split Diagram:

Raw Data (48,000 rows, sorted by date)

|

v

Sort by Date (ascending)

|

v

Cut 1: first 80% / last 20% -----> Test (20%)

|

v

Cut 2 (on remaining 80%): first 80% / last 20%

|

v

Train (64%)

|

v

Validation (16%)

5. Preprocessing & Feature Engineering:

- Data type correction: the date column is parsed to a proper datetime type before any further processing (`DataPreprocessor._fix_dtypes`).
- The missing-value imputer (median for numeric columns, mode for categorical columns) is fitted with `.fit_transform()` on the training split only; validation, test, and December data are only ever passed through `.transform()`, which reuses the training-derived medians/modes.
- The feature engineer's categorical encoders (label encoding, one-hot encoding, and frequency encoding) are fitted with `.fit()` on the training split only; the same fitted encoders are reused via `.transform()` for validation, test data.
- Time-based and distance-based feature construction (`year/month/day_of_week/is_weekend`, `log1p(distance)`) are deterministic functions of the row itself and require no fitting, so they carry no leakage risk regardless of split.

6. Model Training and Validation:

Three regression model families are considered: XGBoost (XGBRegressor), LightGBM (LGBMRegressor), and Random Forest (RandomForestRegressor), all predicting the continuous target `posted_rate`.

6.1 Training and tuning procedure

- **Hyperparameter search:** For each candidate model, Optuna was used to search the hyperparameter ranges defined in `config.yaml`. The optimization used a TPE sampler with a fixed random seed of 42 and 50 trials. XGBoost and LightGBM additionally used Optuna's median-pruning strategy.
- **Tuning objective:** Each Optuna trial was evaluated using the mean RMSE from a 5-fold TimeSeriesSplit cross-validation performed exclusively on the training data. The validation and test sets were not used during hyperparameter optimization.
- **Post-tuning cross-validation:** After selecting the best hyperparameters, each model was independently re-evaluated using a fresh 5-fold TimeSeriesSplit on the training set. Mean RMSE, MAE, and R^2 , together with their standard deviations, were reported to assess model performance and stability.
- **Final training:** Each model was then retrained using the full training split with its selected hyperparameters. The validation set was used for model comparison, while the held-out test set was used for final performance assessment.
- **Model selection:** The model achieving the lowest validation-set RMSE was selected as the final model.

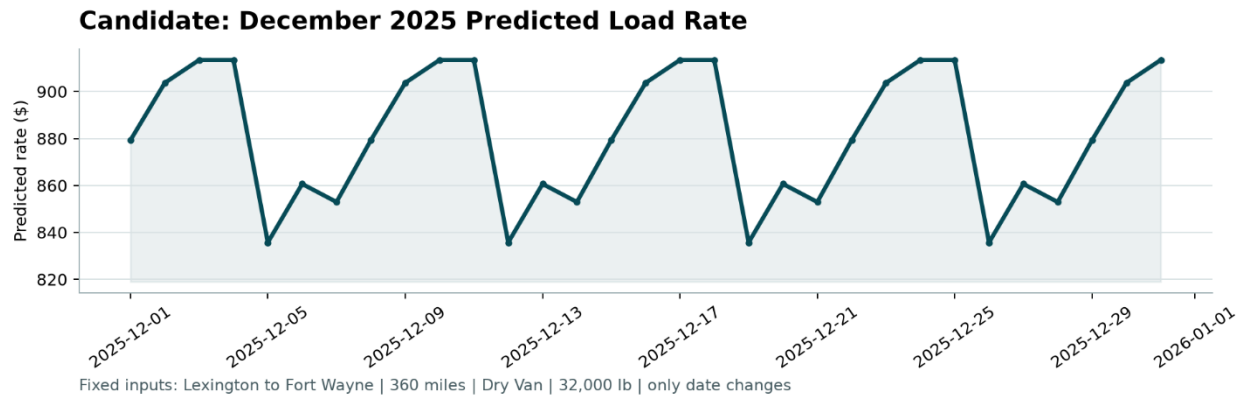
6.2 Validation and test results

Model	Val RMSE	Val MAE	Val R^2	Test RMSE	Test MAE	Test R^2
XGBoost	665.11	230.15	0.803	651.56	220.03	0.817
LightGBM	643.47	138.15	0.816	633.00	124.21	0.828
Random Forest	656.41	160.33	0.808	634.74	129.16	0.827
Best model: LightGBM	643.47	138.15	0.816	633.00	124.21	0.828

LightGBM was selected as the final model for the `validation.csv` submission because it achieved the lowest validation RMSE (643.47) of the three candidates, and it also had the lowest test RMSE (633.00), MAE, and highest R^2 , so the selection is consistent across metrics and not an artifact of picking RMSE alone.

7. Fixed December Prediction:

The fixed December scenario holds every input variable constant except the date, so the resulting predictions isolate whichever purely date-driven pattern (day-of-week and monthly seasonality features) the model has learned, independent of route, weight, or market signals.



- **December Model Generation**

The December models were generated using the same pipeline used for the main models, but with a separate `dec_config.yaml` file. I used a separate configuration because the December data does not contain exactly the same features as the original dataset. It also allows the December processed data and models to be saved separately, so the original artifacts are not overwritten.

In the December configuration, the `drop_optional_features` option can be enabled to remove features that are not available or needed for the December scenario. These include `pickup_lat`, `pickup_lon`, `delivery_lat`, `delivery_lon`, `market_index`, and `quote_signal`. This makes sure that the features produced by the pipeline match the inputs available for the December forecasting task.

The December pipeline still uses the same `run_pipeline()` and `train_model()` functions as the main pipeline. The only difference is that `dec_config.yaml` is passed to both functions.

Therefore, the same main steps are followed: data ingestion, chronological splitting, preprocessing, feature engineering, Optuna hyperparameter tuning, 5-fold TimeSeriesSplit cross-validation, final model training, and model comparison.

Using a separate configuration file also keeps the December processed datasets, models, and other artifacts separate from the original ones. This allowed me to generate the December models without replacing the models already trained on the original dataset.

8. Conclusion:

The dataset was mainly checked in my exploratory notebook for missing values, duplicates, outliers, weight and coordinate ranges, and date range. In the pipeline, I corrected data types and handled missing values using median/mode imputation. I also identified negative and zero weight values, but these were left unchanged.

I used a chronological 64% train / 16% validation / 20% test split to avoid using future data for training. Preprocessing and feature engineering were fitted only on the training data, and TimeSeriesSplit was used for cross-validation to prevent data leakage.

I tested XGBoost, LightGBM, and Random Forest using 50-trial Optuna tuning. LightGBM achieved the lowest validation RMSE and was selected as the best model. For the December forecast, I used a separate dec_config.yaml because the December data has a reduced feature set. The December LightGBM model was then used to predict the 31-day fixed December scenario, where only the date changes. The predictions showed a clear weekly seasonality pattern.